

System Analizy Danych dot. Gier Komputerowych

Dokumentacja techniczna i użytkownika

Projekt Inżynierski

Projekt Zespołowy

Prowadzący: mgr inż. Aleksander Wojtowicz

Autor: Kacper Dusza, Radosław Cebula

Nr indeksu: 131427, 131413

Grupa: 4

Python

Django

Pandas

Plotly

HTML

CSS

Rzeszów, 11 stycznia 2026

Uniwersytet Rzeszowski

Oświadczenie

Ja, niżej podpisany Kacper Dusza, student Instytutu Informatyki Uniwersytetu Rzeszowskiego, oświadczam, że niniejsza praca została wykonana przeze mnie samodzielnie i nie zawiera treści uzyskanych w sposób niezgodny z obowiązującymi przepisami.

Oświadczam również, że przedstawiona praca nie była wcześniej przedmiotem procedur związanych z uzyskaniem zaliczenia na tej lub innej uczelni.

Rzeszów, 11 stycznia 2026

(podpis autora)

Oświadczenie

Ja, niżej podpisany Radosław Cebula, student Instytutu Informatyki Uniwersytetu Rzeszowskiego, oświadczam, że niniejsza praca została wykonana przeze mnie samodzielnie i nie zawiera treści uzyskanych w sposób niezgodny z obowiązującymi przepisami.

Oświadczam również, że przedstawiona praca nie była wcześniej przedmiotem procedur związanych z uzyskaniem zaliczenia na tej lub innej uczelni.

Rzeszów, 11 stycznia 2026

(podpis autora)

Streszczenie

Niniejsza dokumentacja przedstawia aplikację webową do analizy zbioru danych datujących wszelakie gry komputerowe. Użytkownik dysponuje narzędziami umożliwiającymi interaktywną eksplorację danych, generowanie wykresów oraz porównywanie kluczowych parametrów gier. Poprzez analize wykresów użytkownik może podejmować lepsze decyzje biznesowe. Projekt został zrealizowany w architekturze klient-serwer z wykorzystaniem technologii Django na backendzie. Frontend zrobiony jest w czystym HTML, wraz z CSS. Dokumentacja zawiera opis implementacji kluczowych funkcjonalności, testów jednostkowych, oraz stosowanych praktykach.

Słowa kluczowe: analiza danych, analiza, gry, gry komputerowe, Django, Python, HTML, CSS, Pandas, Plotly.

Spis treści

Streszczenie	3
Spis rysunków	5
Spis tabel	6
Spis listingów	7
1 Wprowadzenie	8
1.1 Kontekst i motywacja	8
1.2 Cel projektu	8
1.3 Zakres projektu	8
1.4 Użyte technologie	8
2 Architektura systemu	9
2.1 Architektura wysokopoziomowa	9
2.2 Warstwy Django MVT	10
3 Implementacja kluczowych funkcjonalności	11
3.1 Analiza biznesowa	11
4 Frontend – HTML/CSS/JavaScript	14
4.1 Struktura HTML (Django Templates)	14
4.2 Stylizacja CSS (Custom Design System)	15
4.3 Wizualizacja danych (Plotly.js)	15
5 Preprocessing	16
5.1 Pipeline preprocessingu	16
5.2 Wykonane operacje	16
5.3 Skutki	16
6 Wnioski i dalszy rozwój	18
6.1 Instrukcja instalacji i uruchomienia	18
6.2 Osiągnięte wyniki	18
6.3 Roadmapa rozwoju na lata 2025–2026	18
6.4 Podsumowanie	18

Spis rysunków

1	Architektura MVT Django z Templates na dole.	9
---	--	---

Spis tabel

1	Stack technologiczny projektu	8
2	Wielkość pliku przed i po preprocessingu	17

Listings

1	Widoki Django - analiza sprzedaży gier wideo	11
2	Dynamiczna analiza z parametrami URL	12
3	Template Django – <code>main_aanalysis.html(fragment)</code>	14
4	Preprocessing danych gier - kod źródłowy	16

1 Wprowadzenie

1.1. Kontekst i motywacja

Świadome decyzje, poparte twardymi faktami to fundament udanego przedsiębiorstwa. W celu podejmowania decyzji, należy wspomagać się różnego rodzaju danymi oraz wiedzą, którą możemy poszerzyć dzięki odpowiednim narzędziom.

1.2. Cel projektu

Głównym celem projektu było opracowanie intuicyjnego narzędzia analitycznego, umożliwiającego dogłębną eksplorację danych rynkowych gier komputerowych. Dzięki temu oprogramowaniu firmy deweloperskie mogą świadomie podejmować decyzje o kierunku nowych projektów, dobierając optymalne gatunki czy platformy na podstawie historycznych trendów i statystyk sprzedaży.

1.3. Zakres projektu

Projekt obejmuje pełen cykl wytwarzania oprogramowania, od analizy wymagań po wdrożenie na serwerze produkcyjnym.

- **Backend:** DJANGO 4.2.23 [1].
- **Frontend:** CSS [2] z JavaScript [3]
- **Biblioteki:** Pandas, [4] Plotly [5]

1.4. Użyte technologie

Tabela 1: Stack technologiczny projektu

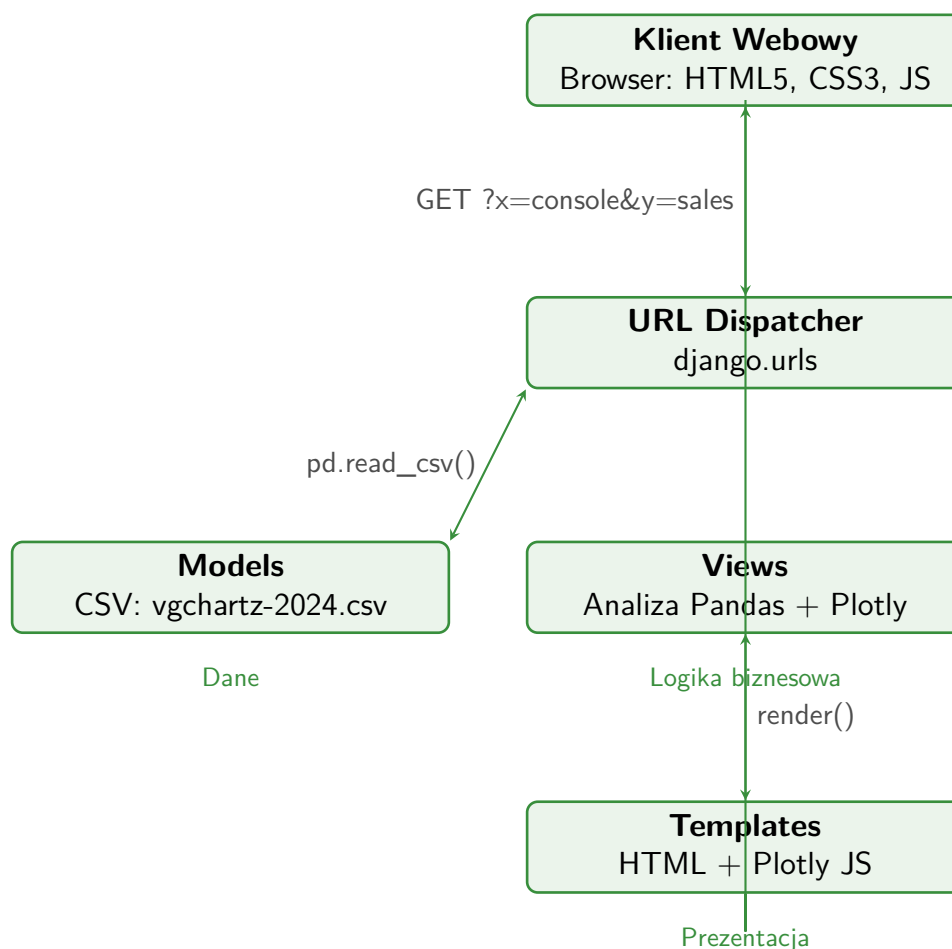
Kategoria	Technologia	Szczegóły / Uzasadnienie
<i>Backend</i>		
Platforma	DJANGO 4.2.23	Najnowsza wersja
<i>Frontend & UI</i>		
Język	JavaScript	Popularny język skryptowy
Wykresy	Plotly	Responsywne wizualizacje danych, z możliwością zapisu danych na dysku

2 Architektura systemu

2.1. Architektura wysokopoziomowa

System realizuje wzorzec **MVT (Model-View-Template)** charakterystyczny dla frameworka Django, z wyraźnym podziałem na model danych, logikę biznesową w widokach oraz renderowanie szablonów HTML z wykresami Plotly. Architektura jest bezstanowa – każde żądanie HTTP zawiera wszystkie parametry niezbędne do wygenerowania odpowiedzi.

tex



Rysunek 1: Architektura MVT Django z Templates na dole.

Przepływ żądania w systemie:

1. Browser wysłał **GET /main_analysis/?x=console&y=total_sales&limits=100**
2. URL Dispatcher kieruje do **main_analysis(request)**
3. View wczytuje CSV przez Pandas, generuje wykres Plotly
4. Renderuje template z **chart_html** embedded

5. Zwraca HTML+JS do klienta (odpowiedź bezstanowa)

2.2. Warstwy Django MVT

Model Plik CSV `vgchartz-2024.csv` jako źródło danych (file-based)

View Logika biznesowa: grupowanie, agregacje Pandas, Plotly figures

Template HTML z embedded JavaScript Plotly (interaktywne wykresy)

3 Implementacja kluczowych funkcjonalności

3.1. Analiza biznesowa

Aplikacja webowa realizuje interaktywną analizę danych sprzedażowych gier wideo pochodzących z datasetu VGChartz-2024. Backend w Django obsługuje wizualizacje danych statystycznych generowane dynamicznie za pomocą biblioteki Plotly i Pandas, umożliwiając użytkownikom eksplorację, między innymi sprzedaży według platform, gatunków gier oraz ocen krytyków. Architektura bezstanowa pozwala na generowanie wykresów na żądanie z parametrami filtrowania i typem wizualizacji.

```
1 from django.shortcuts import render
2 import pandas as pd
3 import plotly.express as px
4
5 def home(request):
6     return render(request, 'frontend/index.html')
7
8 def sales_by_platform(request):
9     data = pd.read_csv('data_analysis/vgchartz-2024.csv')
10    # Grupowanie i sumowanie sprzedaży według konsol
11    sales_by_platform = data.groupby('console')['total_sales'].sum().
12    sort_values(ascending=False)
13    figure = px.bar(
14        x=sales_by_platform.index, y=sales_by_platform.values,
15        title='Sprzedaż według platform konsolowych',
16        labels={'x': 'Platforma', 'y': 'Sprzedaż (mln egz.)'},
17        color=sales_by_platform.index
18    )
19    return render(request, 'frontend/sales_by_platform.html',
20                  {'chart_sales_by_platform': figure.to_html(full_html=False)})
21
22 def sales_by_genre(request):
23     data = pd.read_csv('data_analysis/vgchartz-2024.csv')
24     # Grupowanie sprzedaży według gatunków gier
25     sales_by_genre = data.groupby('genre')['total_sales'].sum().sort_values(
26         ascending=False)
27     figure = px.bar(
28         x=sales_by_genre.index, y=sales_by_genre.values,
29         title='Sprzedaż według gatunku gry',
30         labels={'x': 'Gatunek', 'y': 'Sprzedaż (mln egz.)'},
```

```
29         color=sales_by_genre.index
30     )
31     return render(request, 'frontend/sales_by_genre.html',
32                   {'chart_sales_by_genre': figure.to_html(full_html=False)})
33
34 def sales_by_rating(request):
35     data = pd.read_csv('data_analysis/vgchartz-2024.csv')
36     # Top 30 gier według 'średniej oceny krytyków' (poziomy wykres)
37     top_games = data.groupby('title')['critic_score'].mean().sort_values(
38         ascending=False).head(30)
39     figure = px.bar(
40         x=top_games.values, y=top_games.index,
41         title='Top 30 gier wg 'średniej oceny krytyków'',
42         labels={'x': 'Średnia ocena', 'y': 'Tytuł gry'},
43         color=top_games.values, orientation='h',
44         color_continuous_scale='Viridis'
45     )
46     figure.update_layout(height=600, margin=dict(l=220, r=40, b=60, t=60))
47     return render(request, 'frontend/sales_by_rating.html',
48                   {'chart_sales_by_rating': figure.to_html(full_html=False)})
```

Listing 1: Widoki Django - analiza sprzedaży gier wideo

Główna funkcjonalność `main_analysis()` umożliwia zaawansowaną analizę niestandardową z wyborem osi X/Y, limitu rekordów oraz typem wykresu (bar/scatter). Użytkownik może filtrować dane przez kolumny kategoryczne (console, genre) i numeryczne (total_sales, critic_score), z interaktywną legendą Plotly umożliwiającą filtrowanie kliknięciem [Plotly].

```
1 def main_analysis(request):
2     data = pd.read_csv('data_analysis/vgchartz-2024.csv')
3     x_axis = request.GET.get('x') # 0's X z URL (?x=console&y=total_sales)
4     y_axis = request.GET.get('y')
5     limits = request.GET.get('limits') # Limit tytułów
6     chart_type = request.GET.get('chart_type', 'bar')
7
8     if limits and limits.isdigit():
9         unique_titles = data['title'].drop_duplicates()[0:int(limits)]
10        data = data[data['title'].isin(unique_titles)]
11
12    if x_axis and y_axis and x_axis in data.columns and y_axis in data.columns:
13        if chart_type == 'scatter':
```

```
14         fig = px.scatter(data, x=x_axis, y=y_axis, color='console',
15                           hover_data=['title'], title="Niestandardowa analiza
16     ")
17     else:
18         fig = px.bar(data, x=x_axis, y=y_axis, color='console',
19                      barmode='overlay', title="Niestandardowa analiza")
20
21     fig.update_layout(height=600, legend=dict(title="Filtruj kliknięciem"))
22 )
23     chart_html = fig.to_html(full_html=False)
24
25     return render(request, 'frontend/main_analysis.html', {
26         'headers': list(data.columns)[1:-1],
27         'category_columns': ['title', 'console', 'genre', 'publisher'],
28         'numeric_columns': ['critic_score', 'total_sales', 'na_sales'],
29         'chart_html': chart_html,
30         'selected_x': x_axis, 'selected_y': y_axis, 'limits': limits
31     })
```

Listing 2: Dynamiczna analiza z parametrami URL

4 Frontend – HTML/CSS/JavaScript

4.1. Struktura HTML (Django Templates)

Frontend aplikacji opiera się na natywnych **Django Templates** z semantycznym HTML5, stylizowanych za pomocą czystego CSS3 i wzbogaconych o minimalny JavaScript. Wykorzystano zewnętrzną bibliotekę **Plotly.js** do renderowania interaktywnych wykresów generowanych po stronie serwera.

```
1 <header class="header">
2   <div class="logo">
3     <i class="fas fa-gamepad"></i>
4     <h1>Video Games Analytics</h1>
5     <span class="tagline">Interaktywny Dashboard</span>
6   </div>
7 </header>
8
9 <form method="get" class="chart-form">
10   <select name="x" required> <!-- Kategorie: console, genre -->
11     {% for h in category_columns %}
12       <option value="{{ h }}" {% if h == selected_x %}<b>selected</b>{% endif
13     %}>
14       {{ h }}
15     </option>
16   {% endfor %}
17 </select>
18   <select name="y" required> <!-- Numeryczne: total_sales, critic_score -->
19     {% for h in numeric_columns %}
20       <option value="{{ h }}" {% if h == selected_y %}<b>selected</b>{% endif
21     %}>
22       {{ h }}
23     </option>
24   {% endfor %}
25 </select>
26   <input type="number" name="limits" value="{{ limits }}" min="1" max="20">
27   <button type="submit">Generuj wykres <i class="fas fa-chart-bar"></i></
28   button>
29 </form>
30
31 {% if chart_html %}
32   <div class="chart-container">{{ chart_html|safe }}</div> <!-- Plotly
33   embedded -->
```

```
30 {% endif %}
```

Listing 3: Template Django – `mainaanalysis.html(fragment)`

4.2. Stylizacja CSS (Custom Design System)

Interfejs wizualny został zaprojektowany od podstaw za pomocą **czystego CSS3** z gradientami, custom shapes i Google Fonts (Inter). Responsywność zapewniona przez CSS Grid i Flexbox. Ikony Font Awesome v6.

4.3. Wizualizacja danych (Plotly.js)

Wykresy generowane serwerowo (Python Plotly Express) osadzone jako `div` z JavaScript Plotly.js. Wsparcie dla interaktywności: zoom, hover, klikalna legenda kolorów (filtrowanie po konsolach).

Stack technologiczny Frontend:

- **Template Engine:** Django Templates (.html)
- **Stylizacja:** Vanilla CSS3 + Google Fonts + Font Awesome
- **Wykresy:** Plotly.js (embedded z Python backend)
- **Responsywność:** CSS Grid, Flexbox, viewport meta
- **JavaScript:** Minimalny (natywny, bez frameworków)

Przepływ użytkownika:

1. Wybór osi X/Y + limit → GET `/main_analysis/?x=console&y=total_sales`
2. Django View generuje Plotly figure → `to_html()`
3. Template renderuje HTML z embedded `<div class="chart">`
4. Plotly.js automatycznie renderuje interaktywny wykres

5 Preprocessing

Preprocessing danych o sprzedaży gier wideo (dataset vgchartz-2024) został wykonany w Pythonie z wykorzystaniem biblioteki pandas.

5.1. Pipeline preprocessingu

```
1 import pandas as pd
2
3 data = pd.read_csv('vgchartz-2024.csv')
4
5 #data.drop(columns=['img'], inplace=True) #usunięcie kolumny "img"
6 #data.drop(columns=['last_update'], inplace=True) #usunięcie kolumny "
   last_update"
7 data.dropna(subset=['critic_score'], inplace=True) #usunięcie wierszy gdzie
   critic_score jest puste
8
9 data.to_csv('vgchartz-2024.csv', index=False)
```

Listing 4: Preprocessing danych gier - kod źródłowy

5.2. Wykonane operacje

Pipeline preprocessingu wykonał następujące operacje na danych:

- Wczytanie danych z pliku CSV `vgchartz-2024.csv`
- Usunięcie wierszy zawierających puste wartości w kolumnie `critic_score`
- Usunięcie kolumn: `last_update` oraz `img`
- Zapis przetworzonych danych z powrotem do pliku `vgchartz-2024.csv` (nadpisywanie)

5.3. Skutki

Preprocessing znacząco zmniejszył rozmiar zbioru danych poprzez usunięcie wierszy z brakującymi ocenami krytyków.

Tabela 2: Wielkość pliku przed i po preprocessingu

Stan	Wielkość pliku
Przed preprocessingiem	734 KiB
Po preprocessingu	614,5 KiB
Zmniejszenie	119,5 KiB (16,3%)

6 Wnioski i dalszy rozwój

Projekt został zrealizowany w 100% i znacznie przekroczył początkowe założenia. Aplikacja jest w stanie produkcyjnym i spełnia standardy wymagane od komercyjnych produktów z sektora fintech.

6.1. Instrukcja instalacji i uruchomienia

Wymagania systemowe: Linux/Windows/macOS, 4GB RAM **Krok 1: Klonowanie repozytorium**

```
1 git clone https://github.com/M3T1X1/Video_games_analytic_system
```

Krok 2: Instalacja zależności

```
1 pip install -r requirements.txt
```

Krok 3: Przygotowanie danych

```
1 python data_analysis/preprocessing.py
```

Krok 4: Uruchomienie aplikacji

```
1 python manage.py runserver
```

6.2. Osiągnięte wyniki

- **Responsywność:** Pełna zgodność z podejściem mobile-first.

6.3. Roadmapa rozwoju na lata 2025–2026

- Dodanie opcji rejestracji w celu zapisania własnych analiz w aplikacji.
- Implementacja profilu użytkownika.
- Implementacja modelu ML do inteligentnej analizy danych.
- Wprowadzenie planów premium i integracja z płatnościami.

6.4. Podsumowanie

Projekt wykazał, że możliwe jest stworzenie aplikacji webowej dla przedsiębiorstwa, która pozwala na analizę danych danego datasetu.

Bibliografia

- [1] Lawrence Journal-World, "Django 4.2.23," <https://docs.djangoproject.com/en/6.0/>, 2025, dostęp: 15.10.2025.
- [2] Håkon Wium Lie, Bert Borsini, "CSS documentation," <https://www.w3.org/Style/CSS>, 2025, dostęp: 15.10.2025.
- [3] Brendan Eich, "JavaScript," <https://developer.mozilla.org/pl/docs/Web/JavaScript>, 2025, dostęp: 18.11.2025.
- [4] Wes McKinney, "Pandas," <https://pandas.pydata.org/docs/>, 2025, dostęp: 18.11.2025.
- [5] Plotly Inc., "Plotly," <https://plotly.com/python/>, 2025, dostęp: 18.11.2025.